

Replacing the external LLM reasoner

Can the pipeline’s two LLM-delegated jobs — structured triage and a free-text clinical narrative — be served by self-contained components? Both now ship. Triage is settled and enabled; the narrative ships behind an opt-in flag because the instrumentation still cannot tell a good narrator from a bad one.

Rev. 3 ▪ 12 August 2026

| Teacher: self-hosted Qwen3-8B-AWQ (vLLM)

| Data: 240 RFMiD images

|

Status: deployed and verified live

WHAT CHANGED SINCE REV. 2

Rev. 2 recorded triage as shipped and the narrator as opt-in. Since then the platform has been **deployed to a new Space and verified live**, end-to-end regression and mobile audits have been run against it, and two production defects found by those audits were fixed. §1 answers the original brief directly; §6 is new. The narrator verdict is unchanged, and §4 now carries the measured reason: on the free CPU tier it costs 6–10 s and 427 MB per report, and its small quantized builds cannot run there at all.

Component	Verdict	Basis
Triage — structured decision	Shipped	2 KB model, enabled by default, baked into all three images. Reproduces the 8B teacher exactly; validated out-of-sample on 160 unseen images.
Narrative — grounded template	Shipped (default)	0 MB, deterministic, grounded by construction. Remains the default narrative path.
Narrative — server-hosted LLM	Shipped, opt-in	213.5 MB bf16, published to the Hub and wired behind NARRATOR_ENABLED=false. Three of five blockers remain.
Narrative — on-device / edge	Reachable, not advisable	54.2 MB clears the 60 MB gate, but drops reported findings in 42% of cases. GGUF was tested as the alternative and does not deliver the saving projected for it (§7).

1. Against the brief

The original request set four conditions. All four are met; the table cites the evidence rather than asserting it.

Requirement	Outcome
“Lightweight final model, not much compute for inferencing”	feat_logreg: 3 KB of JSON weights, 0.14 ms on CPU, one matrix multiply. src/triage/ is pure stdlib — no scikit-learn, no numpy, no torch, no network at inference.
“Try other architectures too”	Seven scored on one split (§3). Five reached 1.000; the smallest and fastest was taken. The image CNN scored 0.792 / 0.492 at 6.1 MB and 161 ms.
“Over 75% accuracy and equal or better precision”	1.000 accuracy, 1.000 macro-precision , against a bar of 0.75 and the CNN’s 0.492.
“Test generalizability <i>before</i> creating a docker image”	Done first, on four axes (§3): out-of-sample on 160 fresh unseen images with a different label mix, repeated cross-validation, a learning curve, and an injected EMERGENCY stress test. The image was built only afterwards.

READ THE 1.000 CORRECTLY

The perfect scores mean the *task* is deterministic, not that the model is remarkable. On the out-of-sample set the 8B teacher’s triage equalled the classifier’s own referral_priority in **160 of 160 cases**, so the deterministic rules, a 3 KB linear model and an 8B LLM are indistinguishable here. The value delivered is the removal of an LLM dependency — with its latency, cost and clinical-data egress — from the decision path, not an accuracy gain.

2. What was actually assessed

The pipeline delegates two jobs to an external LLM. They are not the same problem and cannot share a verdict: **triage** is a four-way classification over the disease classifier’s structured output; **narrative** is generation that no classifier can produce.

Everything below rests on measurements taken against a live, self-hosted Qwen3-8B-AWQ teacher on RTX A6000 hardware, not on estimates. Where a figure is arithmetic rather than a measurement, it is labelled.

Table 1: Evidence base. Sample size remains the binding limitation on the narrative half.

Assessment	n	Design	Strength
Triage — out-of-sample	160	Fresh unseen images, different mix	Strong
Triage — cross-validation	80	5-fold × 5 seeds (25 fits)	Strong
Triage — emergency stress	80	Injected CRAO/AION	Synthetic, decisive
Narrative — variant sweep	24	6 variants, one code path	Weak
Narrative — faithfulness	24	Claim-level audit	Weak
Narrative — off-distribution	1	Hand-built single-finding probe	Anecdote

3. Triage — shipped

A 2 KB structured-feature logistic head reproduces the teacher’s priority exactly: accuracy and macro-precision **1.000** held-out, **1.000 ± 0.000** under repeated cross-validation, and **1.000** out-of-sample on 160 previously unseen images with a different label mix. The image CNN it replaces scored 0.792 / 0.492 at 6.1 MB and 161 ms.

WHY IT IS THIS CHEAP

On the out-of-sample set the teacher’s triage equals the classifier’s own `referral_priority` in **160 of 160 cases**. The 8B model adds nothing to the triage *decision*, so a 2 KB linear model, the deterministic rules and an 8B LLM all score identically. This is a cost elimination, not a modelling triumph.

Shipped as: JSON weights rather than a pickle — the model is one matrix multiply, so serving needs neither scikit-learn nor a pickle load at startup. The exporter refuses to write weights that disagree with the fitted estimator on all 240 available cases, and the loader refuses an artifact whose feature layout has drifted. An emergency code escalates through a deterministic override, since EMERGENCY never appeared in training and the head cannot emit it.

4. Narrative — what shipped, and what it costs

Two changes made the narrative half tractable.

Prose-only output. Every reliability defect measured was a malformed-JSON defect, and the JSON existed only to carry the triage fields — which the 2 KB head now serves. A narrator emitting plain prose has no structured contract left to break. That is a stronger guarantee than grammar-constrained decoding (which prevents *invalid* JSON) and needs no extra dependency.

Vocabulary pruning. bitsandbytes leaves the embedding table unquantized, which was the entire size gap: 28.3M embedding params at bf16 (56.6 MB) on top of a 53.1 MB 4-bit body predicts 110 MB, and 109.8 MB was measured. Pruning the vocabulary to the tokens this task can reach — **49,152 → 920** — collapses the embedding to under 1 MB. It is done by id remapping rather than tokenizer surgery, so it is exact and reversible.

Table 2: Compact narrator, 24 held-out cases. Generation rate is 1.000 at every precision — the axis that previously discriminated is now flat, so fidelity decides.

Precision	Size	Gen. rate	Omission	Bad probs	Acuity drift
bf16 (shipped)	213.5 MB	1.000	0.000	0.000	0.250
int8	107.3 MB	37.5 s/case	— evaluation impractical		
nf4	54.2 MB	1.000	0.417	0.167	0.583

TWO REASONS THE NARRATOR SHIPS DISABLED

4-bit trades a visible failure for an invisible one. It clears the 60 MB gate and always produces fluent text — while dropping findings the teacher reported in 42% of cases. Fluent omission is harder to catch than a blank output.

bf16’s clean scores do not generalise. The audit set is dominated by multi-finding cases resembling training data. Given a *single*-finding case by hand, bf16 produced “a 75% de-

tection of diabetic retinopathy with a prevalence of 88%” — for one finding at 88%. Both figures invented. One probe is not a measurement, but it shows the 0.000 belongs to that split rather than to the model.

Why it is disabled on the deployed Space specifically

The Space runs on free cpu-basic (2 vCPU). Measured there, throttled to two threads:

	Value	Against a ~1 s /predict
Model load	8.6 s	once, at startup
Resident memory (fp32)	426.9 MB	fp16 is emulated on CPU, so bf16's 213 MB does not apply
Single-finding report	9.9 s	+10x latency
Three-finding report	6.3 s	+6x latency

Two further constraints make it moot today. `transformers` is **not a declared dependency** anywhere in `pyproject.toml`, so on the deployed image the narrator import fails, the service catches it and serves the template — setting `NARRATOR_ENABLED=true` would change nothing without an image rebuild. And the small quantized builds **cannot run on CPU at all**: bitsandbytes NF4/int8 kernels are CUDA-only, so the 54 MB and 107 MB variants are unavailable on this tier. bf16 is the only option, at ten seconds a report.

5. Blockers — two closed, three open

Blocker	Status	Detail
Structured output best-effort	Closed	Prose-only output removes the parse-failure mode outright. Generation rate 1.000 at every precision.
Compliance statement dropped	Closed	0 of 80 teacher traces carry the AI-disclosure sentence, so it is appended deterministically in <code>src/narrator/service.py</code> rather than trusted to the model.
No serving stack	Open	Still <code>transformers.generate()</code> at batch 1. No defensible latency, throughput or cost model.
Evaluation below clinical standard	Open	One lexical check plus a first-pass claim audit. No NLI/judge cascade, no clinician review.
Statistical power	Open	24 held-out cases, 56 training rows, against ~3,200 available images and a free teacher.

6. Deployment and live verification

The platform is deployed to the Space `landwind22/Retinal_Screening` and is serving: `/health` returns `model_loaded: true` across all 45 diseases. It is the single deployment target — the previous Space

is retired and now points here.

The deploy pipeline used to lie. It reported success as soon as the git push landed, and twice pushed a Space that then failed to start — once from a clobbered README (CONFIG_ERROR — an over-length short_description had also kept the previous Space dark since 23 June), once from a Dockerfile deleted by `rsync -delete (NO_APP_FILE)`. A third deploy was silently dropped when a push-triggered run and a manual dispatch raced to push. All three are fixed: the workflow now polls the runtime stage and requires `/health` to answer before passing, runs are serialised by a concurrency group, and a rejected push refetches and retries. A 6-hourly keep-alive holds the free Space awake against its 48 h sleep timer.

End-to-end regression against the live deployment: `/health`, `/health/gate`, `/api/v1/diseases`, `/api/v1/predict/edge/status` and `/docs` all 200 in ~ 0.73 s, and the fundus gate correctly rejects a non-fundus image with 422 non_fundus_image. The positive prediction path could *not* be exercised: every fundus image in `data/` is a dangling symlink into a cleared cache, and the repository’s own fixture is random noise, which the gate is trained to reject.

Mobile audit (Lighthouse, Moto G Power emulation, throttled 4G — the mid-range Android this product targets):

	Before	After	
Performance	80	95	
Accessibility	96	100	
Largest Contentful Paint	5.0 s	2.6 s	target 2.5 s
Transferred payload	776 KB	293 KB	–62%
Colour-contrast failures	31	0	
Redundant-alt images	2	0	

Two defects, both real. **Text compression was off:** 481 KB of 776 KB shipped uncompressed, costing 2.4 s on 4G. The fix needed `gzip_proxied` any — nginx reverse-proxies Next.js and uvicorn, and will not compress proxied responses without it, which is exactly where the payload was. **Contrast** failed on 31 nodes; the largest group was not a bad colour but `opacity-70` blending a passing slate-500 down to 2.71:1. Colour values were recomputed per background — on dark sections the correction runs lighter, not darker, a distinction that cost one regression before it was caught.

Lighthouse scores move ~ 5 points run to run on a shared runner; a single number should not be read as a trend.

7. Format evaluation — AWQ, GPTQ, GGUF

Rev. 1 recommended AWQ/GPTQ over bitsandbytes. Evaluating that properly reverses the recommendation, for a reason specific to this model: a 135M SmolLM2 is **21% embedding**, and what each format does with that table decides the footprint.

Table 3: AWQ/GPTQ rows remain arithmetic — neither library would install here. The GGUF rows are **measured**: converted with `convert_hf_to_gguf.py` and quantized with `llama-quantize` built from source.

Format	Body	Embedding	Total
bnb NF4	4-bit	fp16, not quantized	109.8 MB
bnb int8	int8	fp16, not quantized	107.3 MB
AWQ / GPTQ (W4A16)	4-bit	fp16, conventionally excluded	~110 MB
GGUF Q4_K_M	mixed, 6.17 BPW	q8_0	105.5 MB
GGUF Q5_K_M	mixed	q8_0	112.1 MB
GGUF Q8_0	q8_0	q8_0	144.8 MB

AWQ and GPTQ would not beat bitsandbytes on size here. They exclude the embedding exactly as bnb does, landing at the same ~110 MB; their advantage is throughput via fused kernels, which does not address the binding constraint.

The GGUF projection in Rev. 2 was wrong: 105.5 MB measured against ~80 MB projected, which leaves it barely smaller than the 109.8 MB bitsandbytes build rather than the step change the arithmetic implied. Two architecture-specific facts the projection ignored:

1. **The embedding is quantized — but to 8-bit, not 4-bit.** `token_embd.weight` converts f16 -> q8_0, 54.00 MiB to 28.69 MiB. The central claim (GGUF is the only one of the three that quantizes the embedding at all) *holds*; the saving is 1.9×, not the 4× assumed.
2. **SmolLM2’s hidden size is 576, which is not divisible by 256.** `q4_K` requires that, so **180 tensors fell back** to larger types (166 to q5_0, 14 to q8_0). `llama.cpp`’s own summary reports the outcome: `quant size = 98.87 MiB (6.17 BPW)`, against the ~4.5 BPW assumed.

That is a property of *this* model’s width, not of GGUF. A model whose hidden size is a multiple of 256 would avoid the fallback, so the projection may hold elsewhere — but it does not hold here, and **no format tested reaches the 60–100 MB band without vocabulary pruning.**

Fidelity under GGUF remains unmeasured. `llama-cli` hung on this host even on a trivial prompt, so the open question — whether `Q4_K_M` avoids the 42% omission that made the pruned 4-bit build unusable — is still open. At 105.5 MB it would in any case compete with bf16 at 213 MB rather than offer an edge-sized option.

Pruning and standard formats are alternatives, not complements. The pruned checkpoint deliberately breaks the model/tokenizer correspondence (`vocab_size` 920 against a 49,152-token tokenizer, bridged by an explicit id remap). GGUF bakes the tokenizer into the file — confirmed here, since `convert_hf_to_gguf.py` requires `vocab_size` to equal the tokenizer size, which the pruned checkpoint violates (920 vs 49,152); AWQ/GPTQ would run mechanically but still need the custom wrapper, losing the vLLM/SGLang serving compatibility that justifies adopting them.

Tooling status (measured). `autoawq` 0.2.9 installs but cannot import against `transformers` 4.57.1 and is unmaintained upstream; `gptqmodel` and `llmcompressor` require `torch` ≥ 2.8; `compressed-tensors` 0.11.0 is present as a vLLM dependency; `llama.cpp` binaries (`llama-quantize`, `convert_hf_to_gguf.py`) are absent, and the `gguf` Python library alone writes only F16, not K-quants.

8. Recommendation

Triage: done and deployed. It is validated, effectively free, removes an LLM dependency from the decision path outright, and is live in all three images with the Space verified serving.

Narrative: keep the template as the default and leave `NARRATOR_ENABLED=false`. The bf16 narrator is shipped and usable for evaluation, but nothing has changed about the reason not to enable it: the instrumentation still cannot distinguish a good narrator from a bad one, and in a clinical screening context that is the thing that has to work first. The off-distribution probe (§3) is a concrete reminder that clean held-out numbers are not evidence of general reliability.

Next, in order: (1) regenerate traces at scale with the disclosure mandated — it closes the statistical-power and compliance gaps together and the teacher is free; (2) build a real faithfulness evaluation with clinician review; (3) try GGUF Q4_K_M for the 60–100 MB target rather than more bitsandbytes; (4) measure under vLLM before quoting any latency. Only then revisit enabling the narrator by default.

The honest summary is unchanged from Rev. 1: the expensive component turned out to be unnecessary for triage, and not yet trustworthy for narrative. The first is now banked.

Sources.

docs/28-reasoner-cnn-vs-distilledqwen.md

docs/29-narrator-verification-and-gaps.md

outputs/reasoner_comparison_real/ outputs/generalizability/

Provenance. Measurements on RTX A6000 against a live Qwen3-8B-AWQ teacher. Quantization-format sizes in §6 are arithmetic, not measurements, and are labelled as such. Effort estimates carry no measurement behind them.